

INFORME DE LA PRÁCTICA – ToDo List Desacoplada

1. Introducción

El objetivo de esta práctica es desarrollar una aplicación backend aplicando principios fundamentales de arquitectura de software, como la separación por capas, la abstracción y el desacoplamiento.

La aplicación consiste en una API REST que gestiona una lista de tareas (To-Do List). En esta versión, los datos se almacenan temporalmente en memoria, pero el sistema está diseñado para permitir la integración futura de una base de datos real sin modificar las capas superiores.

El backend se ha desarrollado en Java usando Javalin, y el frontend se ha generado con Vue 3 y Axios.

2. Arquitectura del Sistema

La aplicación se organiza en tres capas bien definidas y completamente independientes entre sí:

2.1. Capa de Presentación (Controlador)

Es la capa encargada de recibir las peticiones HTTP y devolver respuestas en formato JSON.

No contiene lógica de negocio; simplemente valida la petición y delega el procesamiento al servicio.

2.2. Capa de Servicio (Lógica de Negocio)

Aquí se encuentran las reglas de negocio y las validaciones necesarias.

Gestiona la creación, lectura, actualización y borrado de tareas, asegurándose de que los datos sean válidos y que las operaciones tengan sentido.

Esta capa no tiene conocimiento de cómo se almacenan los datos.

2.3. Capa de Datos (Repositorio)

Define las operaciones necesarias para interactuar con la fuente de datos mediante una interfaz.

La implementación actual utiliza una estructura en memoria para simular una base de datos, pero puede sustituirse fácilmente por una implementación real como MySQL o PostgreSQL.

3. Modelo de Dominio

El modelo principal de la aplicación es la entidad “Task”, que representa una tarea dentro de la To-Do List. Contiene atributos como identificador, título, descripción y un indicador de si la tarea está completada.

4. Repositorio (Contrato y Implementación)

4.1. Interfaz de Repositorio

Se define una interfaz que especifica las operaciones necesarias para gestionar las tareas. Incluye acciones como guardar, obtener todas las tareas, buscar por identificador, eliminar y actualizar.

La existencia de esta interfaz asegura que la capa de servicio dependa de una abstracción y no de una implementación concreta.

4.2. Implementación en Memoria

La implementación actual utiliza una estructura en memoria RAM para almacenar las tareas durante la ejecución. Esta implementación es temporal, pero demuestra el concepto de que la aplicación no está acoplada a un tipo de almacenamiento específico.

5. Capa de Servicio

La capa de servicio recibe la interfaz del repositorio como dependencia.

Se encarga de aplicar la lógica necesaria, como validar que los títulos no estén vacíos, gestionar la actualización de estados y asegurar que las tareas existan antes de modificarlas o eliminarlas.

Su principal característica es que trabaja exclusivamente con la interfaz, por lo que es completamente independiente del tipo de almacenamiento utilizado.

6. Controlador REST

El controlador expone la API REST mediante rutas HTTP. Gestiona operaciones como obtener todas las tareas, crear una nueva, marcarlas como completadas y eliminarlas.

Todas las operaciones devuelven respuestas en formato JSON.

El controlador depende solamente de la capa de servicio, y nunca accede directamente a la capa de datos.

7. Inyección de Dependencias

La aplicación utiliza inyección de dependencias manual en el punto de entrada, donde se decide qué implementación del repositorio se utilizará.

Esta técnica asegura que la lógica de negocio y la capa de presentación no tengan conocimiento de la implementación concreta del almacenamiento.

Si en un futuro se implementa un repositorio basado en base de datos real, solo será necesario sustituir la instancia en el punto de entrada sin modificar ninguna otra parte del sistema.

8. Frontend (Vue 3 + Axios)

El frontend fue generado utilizando Vue 3. Permite visualizar las tareas, crear nuevas, marcarlas como completadas y eliminarlas.

Se comunica con el backend mediante peticiones HTTP enviadas a la API REST.

El archivo del frontend se sirve directamente desde el mismo servidor backend, permitiendo una integración rápida y sencilla.

9. Justificación del Uso de Interfaces

El uso de la interfaz del repositorio es fundamental para lograr el desacoplamiento entre la lógica de negocio y la implementación del almacenamiento.

Gracias a esta abstracción, el servicio no necesita conocer cómo se guardan los datos, lo que permite:

- Sustituir la fuente de datos sin modificar las capas superiores.
- Facilitar las pruebas unitarias mediante implementaciones simuladas.
- Cumplir principios SOLID, especialmente el principio de inversión de dependencias.
- Hacer la aplicación más mantenible, flexible y escalable.

10. Conclusión

La práctica demuestra el diseño de una arquitectura desacoplada y profesional, siguiendo buenas prácticas de ingeniería de software.

El uso de capas, interfaces e inyección de dependencias permite crear una aplicación modular y fácil de extender.

El sistema queda preparado para evolucionar hacia una infraestructura real sin necesidad de cambiar la lógica principal.